

End-Term Assignment

Signal, Systems and Networks

EE200

Instructor: Dr. Tushar Sandhan

Submitted by: Lalit Meena

Roll No.: 240591

Q1) Frequency Mixer: ‘Beauty and the Blur’

1 Introduction

This exercise delves into how our visual system deciphers images through spatial frequency analysis. High spatial frequencies reveal fine details like edges, while low frequencies capture the broader structure and shape. This concept is exemplified by hybrid images, where perception shifts with distance—for instance, recognizing Einstein up close and Marilyn Monroe from afar.

Through 2D Discrete Fourier Transform (DFT), the exercise involves visualizing frequency components of facial images, examining how rotation affects the frequency domain, and creating a frequency-based image mixer that blends the detail of one image with the structure of another using Python.

2 Aim

This exercise focuses on analyzing and manipulating images in the frequency domain using Python. The goal is to visualize frequency spectra, explore the impact of spatial transformations, and develop a frequency mixer that blends two images—one contributing high-frequency details and the other low-frequency structure.

- Perform 2D Discrete Fourier Transforms (DFT) on the input images.
- Create and apply frequency-domain filters such as low-pass and high-pass masks.
- Merge the filtered frequency components and reconstruct the final hybrid image.

3 Input

- For high frequency characteristics, i.e. details in the fused image, the cat image is chosen.
- For low frequency characteristics, i.e. the overall outline/structure in the fused image, the dog image is chosen.



Figure 1: Cat image



Figure 2: Dog image

4 Theory

• Fourier Transform

The Fourier Transform is a foundational tool in signal processing that decomposes signals into their frequency components. In image processing, the 2D Discrete Fourier Transform (DFT) allows us to analyze spatial frequency content by converting images into the frequency domain.

Low frequencies capture global structure and smooth variations, while high frequencies correspond to sharp transitions and fine details.

Mathematically, for an image $I(x, y)$ of size $M \times N$, the 2D DFT is defined as

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

The inverse 2D DFT to reconstruct the image is as follows:

$$I(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

The frequency-domain result $F(u, v)$ is a complex-valued function and characterized by:

- Magnitude $|F(u, v)|$ — indicates the intensity of each frequency component.
- Phase $\angle F(u, v)$ — encodes spatial positioning within the image.

To compute the 2D DFT in practice, we use the `fft2` function from Python's `numpy.fft` module, which efficiently transforms an image from the spatial to the frequency domain.

- **Magnitude Spectra**

The magnitude spectrum is defined as the absolute value of the Fourier transform:

$$|F(u, v)| = \sqrt{(\text{Re}(F(u, v)))^2 + (\text{Im}(F(u, v)))^2}$$

Due to the wide dynamic range, visualizing this directly often produces dark images. To resolve this, we apply a logarithmic scale.

$$|F(u, v)|_{dB} = 20 \log_{10} (|F(u, v)| + \epsilon)$$

where

$$\epsilon = 10^{-5}$$

prevents logarithmic singularities and improves visibility of all frequency components.

Two types of magnitude spectra were used for visualization:

- **Normal Magnitude Spectrum**

The normal (or linear) magnitude spectrum represents the absolute values of the 2D Fourier transform directly without any scaling. Although it accurately reflects the distribution of frequency components, the wide range of values causes most high-frequency details to appear very dim.

- **Logarithmic (dB) Magnitude Spectrum**

The logarithmic magnitude spectrum compresses the dynamic range and is calculated using

$$|F(u, v)|_{dB} = 20 \log_{10} (|F(u, v)| + \epsilon)$$

This scaling allows finer details in the frequency domain to become visible.

The above analysis for cat and dog is shown below:

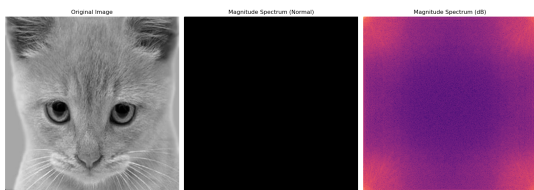


Figure 3: Cat Frequency Analysis

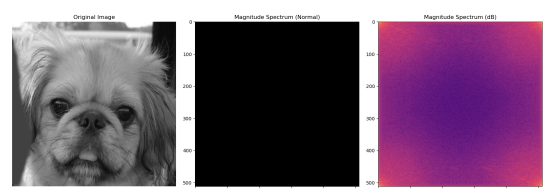


Figure 4: Dog Frequency Analysis

5 Libraries Used in Code

- NumPy
- Matplotlib
- Pillow

6 Spectrum Centering

In the 2D Discrete Fourier Transform (DFT), low-frequency components representing the overall structure or average intensity are initially positioned at the corners of the frequency spectrum.

To improve interpretability, these components can be shifted to the center of the spectrum using the `fftshift` function from Python's `numpy.fft` module.

Centering the spectrum places low frequencies at the center and high frequencies toward the edges, offering a more intuitive visualization of the spatial frequency distribution.

7 Effect of Rotating Image

To analyze the behavior of the frequency-domain representation, we rotated one of the images by 90° counter-clockwise and then computed its 2D Fourier Transform using the same steps: applying `fft2`, centering the spectrum with `fftshift`, and using logarithmic scaling for visualization.

Observations:

- The overall structure of the magnitude spectrum remains unchanged after rotation.
- Each feature in the frequency-domain representation is rotated by exactly 90° compared to the original spectrum.
- This experiment confirms a fundamental property of the 2D Fourier Transform: a 90° counter-clockwise rotation in the spatial domain results in an equivalent 90° rotation in the frequency domain while preserving the magnitude distribution.

““latex

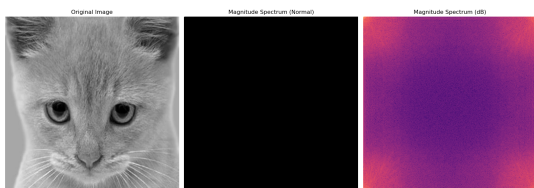


Figure 5: Before centering

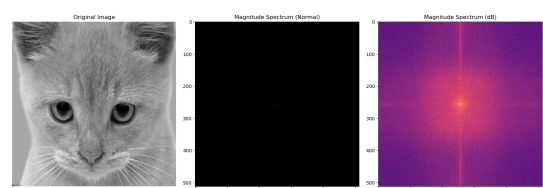


Figure 6: After centering

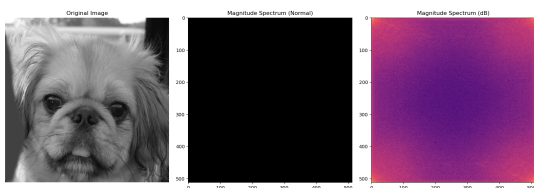


Figure 7: Before centering

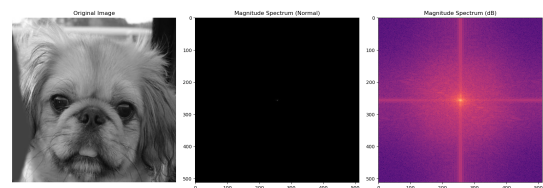


Figure 8: After centering

- Each feature in the frequency-domain representation is **rotated** by exactly 90° compared to the original spectrum.
- This experiment confirms a fundamental property of the 2D Fourier Transform: a 90° counter-clockwise rotation in the spatial domain results in an equivalent 90° rotation in the frequency domain, while preserving the magnitude distribution.

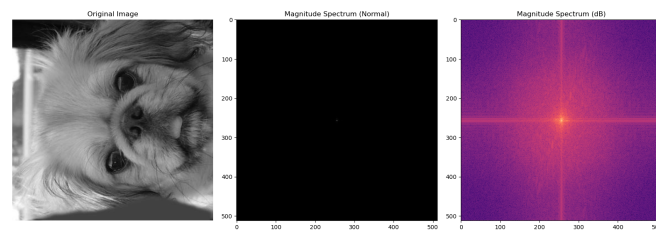


Figure 9: Rotaing Dog

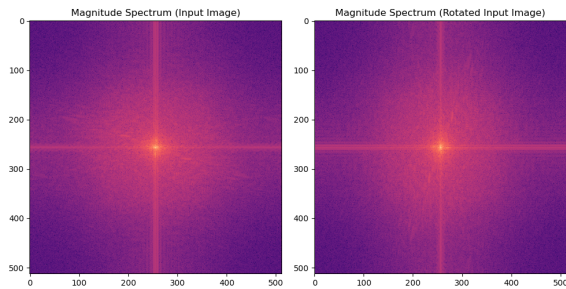


Figure 10: Comparison before and after rotating

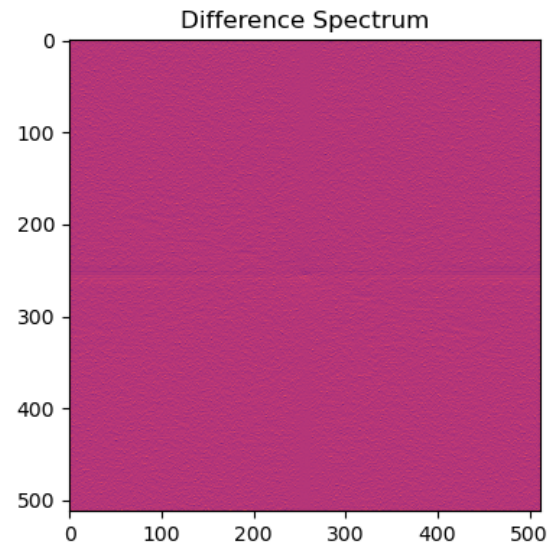


Figure 11: Difference

8 Results on Fusing

The objective of the frequency mixer is to combine the low-frequency components from one image—which contribute to the overall structure—with the high-frequency components from another image—which provide fine textural details.

- The dog image serves as the source of low-frequency components, capturing the broad shapes and structures.
- The cat image provides the high-frequency components, highlighting edges and fine textures.

Transfer Function (Masks):

- **Low-pass mask:** A circular binary mask that retains only the low-frequency components near the center of the shifted spectrum, filtering out high-frequency details.
- **High-pass mask:** The complement of the low-pass mask, preserving high-frequency components while suppressing the low-frequency content.

Approach:

- A binary circular mask was employed to separate low- and high-frequency components, generated using a custom function.
- This function returns a boolean mask with True values inside a specified radius from the center of the frequency spectrum and False values elsewhere.
- Various radius values (e.g., 10, 20, 30) were tested empirically to evaluate the trade-off between retaining coarse structure and preserving fine details.

- A radius of approximately 14 pixels was found to offer a good balance—preserving overall shape in the low-pass component and capturing textures in the high-pass component.
- Additionally, weights were introduced after masking the Fourier transforms to fine-tune the contribution of each frequency band. These were determined through iterative experimentation based on the visual quality of the fused image.

Implementation Steps:

- Computed the 2D DFT of both images using `fft2`, followed by centering the spectrum with `fftshift`.
“ “`latex`
- Created circular masks centered at $(N/2, N/2)$, where N is the dimension of the frequency domain.
- Defined the low-pass region using a circular area of radius r .
- Derived the high-pass mask as the complement of the low-pass mask.
- Applied the masks to the respective spectra of the images, combined them (with appropriate weights), and reconstructed the fused image using `ifftshift` and `ifft2`.

Let me know if you'd like help adding diagrams or visualizations of the masks and results.

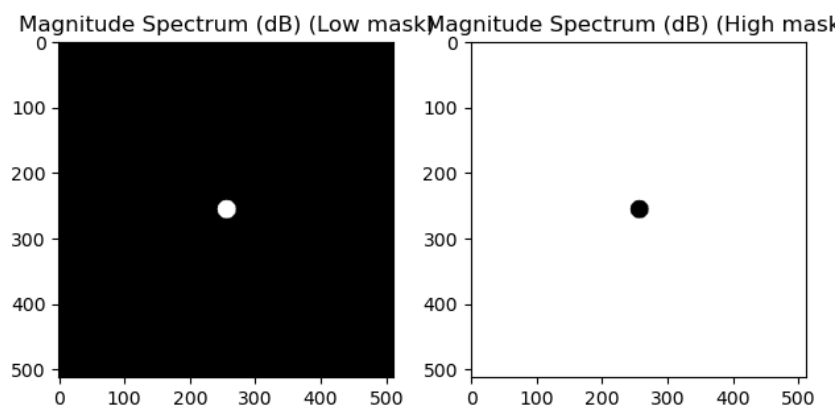


Figure 12: Magnitude Spectra of Masks



Figure 13: Resulting Image

Q2) Frequency De-Mixer: ‘Unwanted Solo’

9 Introduction

This exercise tackles the issue of intrusive instrumental beats that distort the harmonic structure of a song. Such disruptions—often resulting from production flaws or post-processing artifacts—can detract from the intended musical experience.

To mitigate this, we utilize frequency-domain signal processing to identify and isolate these unwanted elements based on their unique spectral characteristics. By crafting and applying precise filters, the goal is to effectively suppress the interference while maintaining the quality and authenticity of the original audio.

10 Aim

The primary objective of this project is to develop a frequency-selective filtering system—termed a frequency demixer—to suppress unwanted instrumental elements from a corrupted audio signal.

The specific goals are as follows:

- Identify and isolate the frequency band associated with the undesired solo instrumental component.
- Design and implement appropriate digital filters (e.g., band-stop, high-pass, or low-pass) in Python to effectively attenuate the interfering frequencies.
- Analyze the characteristics and effectiveness of the designed filters using tools such as Bode plots, Power Spectral Density (PSD), and pole-zero plots.
- Reconstruct the audio signal with minimal distortion, aiming to restore the original musical content as accurately as possible.

11 Input

We are given a music track that has been corrupted by the presence of unwanted instrumental beats, which disrupt the intended harmonic structure.

As we analyze the audio, our objective is to identify and isolate these intrusive elements and apply appropriate signal processing techniques to suppress them while preserving the integrity of the original music.

12 Libraries Used in Code

- NumPy
- Matplotlib
- Librosa
- Scipy.signal

- Soundfile
- Polynomial

13 Theory

- **Audio Signal Analysis**

The signal was first examined in the time domain by plotting its waveform, which revealed variations in amplitude over time.

To analyze its spectral content, the magnitude spectrum was computed and visualized using the Fourier Transform, providing a frequency-domain perspective.

To further understand how the frequency content changes over time, a spectrogram was generated.

This time-frequency representation helped highlight dominant frequency bands and areas potentially affected by noise.

These initial visualizations laid the groundwork for designing targeted filters aimed at modifying or eliminating specific frequency components from the signal.

Observations from the Analysis

- The time-domain waveform exhibited complex amplitude patterns with prominent peaks, suggesting the presence of a dominant, possibly intrusive, audio element.
- The spectrogram (on a logarithmic scale) revealed a bright, narrow horizontal band between 1024 Hz and 2048 Hz, consistently present throughout the track.
- Other frequency regions showed moderate to low energy, likely corresponding to background vocals or instrumental layers.
- The magnitude spectrum displayed a clear concentration of energy between 1100 Hz and 1800 Hz, sharply separated from adjacent frequency bands.
This range likely corresponds to an added solo instrumental component not originally intended in the mix.

- **Filter Design and Implementation**

A series of frequency-selective filters were employed to analyze and manipulate various components of the audio signal.

Each filter played a distinct role in isolating or suppressing specific elements, enhancing our understanding of the signal's spectral structure and informing noise reduction strategies.

Low-Pass Filter

- A low-pass filter with a cutoff frequency of 1150 Hz was applied, effectively removing most high-frequency content from the audio.
- The resulting output was dominated by vocal elements, while high-frequency instruments—including the intrusive flute—were significantly attenuated.

- This suggests that vocal components are primarily concentrated below 1000 Hz, whereas the unwanted flute lies in the higher-frequency spectrum, validating the targeted frequency band for suppression.

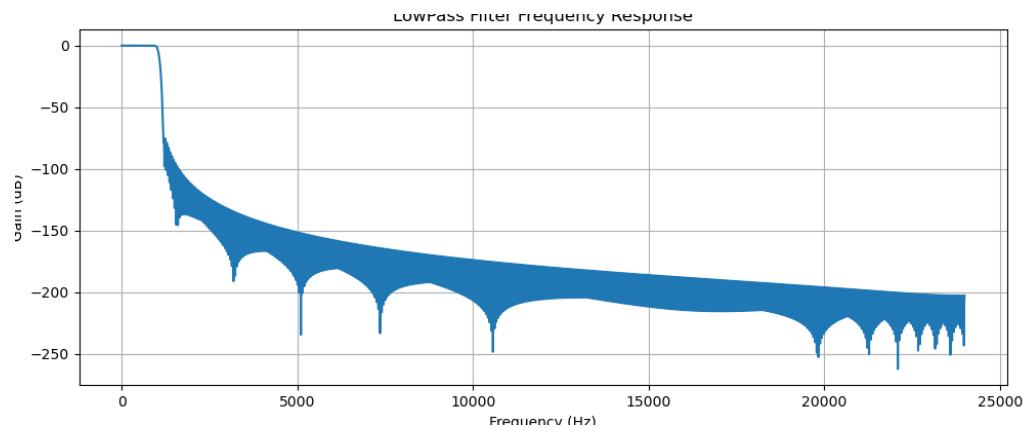


Figure 14: Lowpass filter frequency response

High-Pass Filter

- A high-pass filter with a cutoff at 1700 Hz was applied to retain high-frequency components, including harmonic content and portions of the piccolo flute signal.
- The filtered output preserved only the instrumental elements, while lower-frequency vocal components were effectively suppressed.
- This confirms that the intrusive piccolo flute and other high-frequency noise are located above 1700 Hz, supporting the case for a high-frequency noise removal approach.

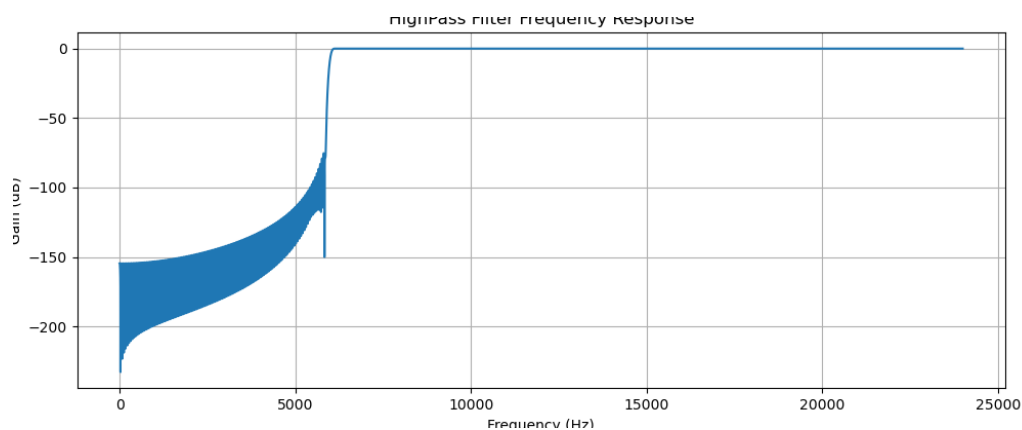


Figure 15: Highpass filter frequency response

Band-Stop Filter

- To suppress a broader mid-frequency range, a band-stop filter was applied between 1050 Hz and 6000 Hz.
- This significantly reduced the presence of the intrusive flute while preserving vocals and background music with minimal distortion.
- These results indicate that the selected frequency band effectively isolates the unwanted component, demonstrating that band-stop filtering can improve audio quality without sacrificing essential musical content.

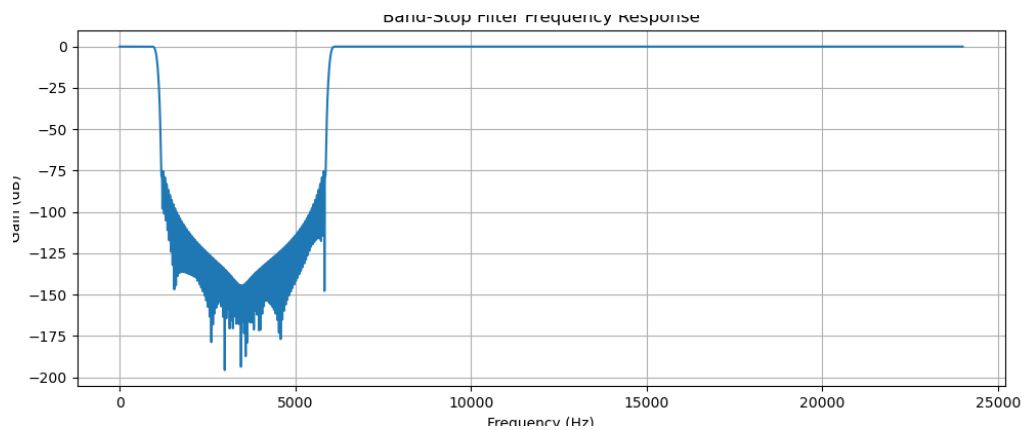


Figure 16: Bandpass filter frequency response

Conclusion

- Each filter provided valuable insights into the spectral composition of the audio.
- Among them, the band-stop filter proved most effective for noise suppression, achieving a favorable trade-off between eliminating unwanted content and maintaining audio fidelity.

““

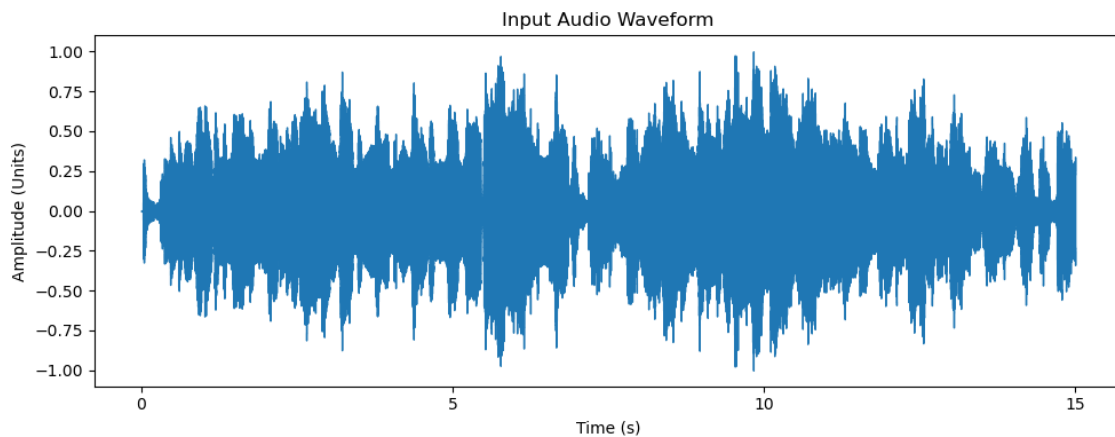


Figure 17: Time-Domain Waveform of the Original Audio

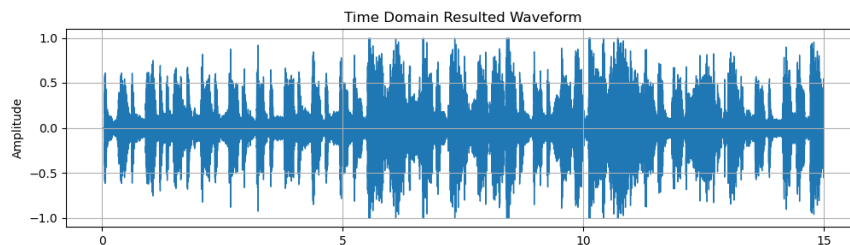


Figure 18: Time-Domain Waveform of the Filtered(Output) Audio